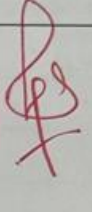


SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	S.M.SNEHA.	Reg.No.:	192321095
Department:	IT	Date:	

(16.1.18) 

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I S.M. SNEKA (192321095) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sneka.

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)	
Q1	3	3	2	3	3	14
Q2	4	3	2	3	2	14
Q3	3	4	3	4	2	16
Q4	2	3	4	3	3	15
Q5	3	3	5	3	3	17

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)

76

1)A processor performs arithmetic operations on signed 8-bit integers using 2’s complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Problem Understanding: The processor uses 8-bit 2’s complement representation. The two operands are +45 and −23. Both addition and subtraction must be performed within the 8-bit signed range −128 to +127.

Binary Conversion:

+45 = 0010 1101

+23 = 0001 0111

−23 = 1110 1001

(invert 0001 0111 → 1110 1000, then +1)

Addition: +45 + (−23)

0010 1101 (+45)

+ 1110 1001 (−23)

1101 0110

The result 1101 0110 has MSB = 1, so it is negative. Its magnitude is obtained by taking 2’s complement: 0010 1010 = 42. Therefore, the result is −42.

Subtraction: +45 − (−23) = +45 + (+23)

0010 1101 (+45)

+ 0001 0111 (+23)

0100 0100

0100 0100 = 68 in decimal. Therefore, +45 − (−23) = +68.

Overflow Detection: In 2’s complement addition, overflow occurs when two numbers having the same sign produce a result with the opposite sign. For +45 + (−23), the operands have different signs, so overflow cannot occur. For +45 + +23, both operands are positive and the result is +68, which is still within −128 to +127. Hence, no overflow occurs in either operation.

Engineering Interpretation: The processor detects signed overflow from the carry into the sign bit XOR the carry out of the sign bit. Since this condition is false for both operations, the final 8-bit results are valid: −42 for addition and +68 for subtraction.

2)A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Problem Understanding: A CLA reduces addition delay by calculating carry signals in parallel instead of waiting for each carry to ripple through a full adder. For illustration, consider A = 1011 and B = 0110 with input carry C0 = 0.

Generate and Propagate Signals:

For each bit i: $P_i = A_i \text{ XOR } B_i$, $G_i = A_i \text{ AND } B_i$

Bit: 3 2 1 0

A: 1 0 1 1

B: 0 1 1 0

P: 1 1 0 1

G: 0 0 1 0

Carry equations:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

With $C_0 = 0$:

$$C_1 = 0, C_2 = 1, C_3 = 1, C_4 = 1$$

$$S_0 = A_0 \text{ XOR } B_0 \text{ XOR } C_0 = 1$$

$$S_1 = A_1 \text{ XOR } B_1 \text{ XOR } C_1 = 0$$

$$S_2 = A_2 \text{ XOR } B_2 \text{ XOR } C_2 = 0$$

$$S_3 = A_3 \text{ XOR } B_3 \text{ XOR } C_3 = 0$$

Result $C_4 S_3 S_2 S_1 S_0 = 1\ 0001 = 17$ decimal

Comparison with Ripple Carry Adder: In a ripple carry adder, each full adder generally waits for the carry from the previous stage. For an n-bit adder, the worst-case carry delay increases approximately linearly with n. A CLA forms the carry equations in parallel, so a 4-bit CLA has much smaller carry propagation delay. Modern processors use carry-look-ahead and related parallel-prefix techniques because faster carry generation directly reduces arithmetic execution time and improves overall CPU performance.

3)In a signed multiplication operation, a processor uses Booth’s algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Problem Understanding: Booth’s algorithm performs signed multiplication using 2’s complement arithmetic. Use 4-bit operands and four iterations. Multiplicand $M = -6 = 1010$, multiplier $Q = +5 = 0101$, accumulator $A = 0000$, and Booth bit $Q_{-1} = 0$.

Booth Rules:

$Q_0 Q_{-1} = 01 \rightarrow A = A + M$

$Q_0 Q_{-1} = 10 \rightarrow A = A - M$

$Q_0 Q_{-1} = 00 \text{ or } 11 \rightarrow \text{No arithmetic operation}$

After the operation, perform an arithmetic right shift on $[A, Q, Q_{-1}]$.

Cycle	$Q_0 Q_{-1}$	Operation	A after operation	After shift: A	After shift: $Q \mid Q_{-1}$
1	10	$A = A - M$	0110	0011	0010 1
2	01	$A = A + M$	1101	1110	1001 0
3	10	$A = A - M$	0100	0010	0100 1
4	01	$A = A + M$	1100	1110	0010 0

Final Product:

$A \ Q = 1110 \ 0010$

1110 0010 is negative in 8-bit 2’s complement.

Invert: 0001 1101; $+1 = 0001 \ 1110 = 30$

Therefore, $(-6) \times (+5) = -30$.

Why Booth’s Algorithm is Efficient: Booth encoding processes runs of consecutive 1s in the multiplier as a single add/subtract transition instead of performing a separate addition for every 1-bit. It naturally supports signed 2’s complement operands and uses arithmetic right shifts to preserve the sign. This can substantially reduce arithmetic operations when the multiplier contains long runs of 1s.

4)A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Problem Understanding: The dividend is $13 = 1101_2$ and divisor is $3 = 0011_2$. A 4-bit quotient is produced. The expected result is quotient = 4 and remainder = 1.

A. Restoring Division

Initial values: $A = 00000$, $Q = 1101$, $M = 00011$. At every iteration, $[A,Q]$ is shifted left. M is subtracted from A . If A becomes negative, the previous A is restored and quotient bit Q_0 is set to 0; otherwise Q_0 is set to 1

Step	A after shift	A – M	Restored/Kept A	Q after step
1	00001	$00001 - 00011 = -2$	Restore → 00001	1010
2	00011	$00011 - 00011 = 0$	Keep → 00000	0101
3	00000	$00000 - 00011 = -3$	Restore → 00000	1010
4	00001	$00001 - 00011 = -2$	Restore → 00001	0100

Final restoring result: $Q = 0100_2 = 4$, $A = 00001_2 = 1$

Hence, $13 \div 3$ gives quotient 4 and remainder 1.

B. Non-Restoring Division

Initial values: $A = 00000$, $Q = 1101$, $M = 00011$. Instead of restoring A immediately after a negative result, the next iteration compensates by adding M when A is negative.

Step	Sign before shift	Operation	A after operation	Q after step
1	$A \geq 0$	$A = A - M$	-2	1010
2	$A < 0$	$A = A + M$	0	0101
3	$A \geq 0$	$A = A - M$	-3	1010
4	$A < 0$	$A = A + M$	-2	0100

At the end, A is negative (-2), so a final correction is performed: $A = A + M = -2 + 3 = 1$.

Final non-restoring result: $Q = 0100_2 = 4$, $A = 00001_2 = 1$

Comparison: Both methods require one iteration per quotient bit, so both use four main iterations for this 4-bit example. Restoring division may perform an extra restore operation whenever subtraction produces a negative remainder. Non-restoring division avoids this immediate restoration and uses the sign of the partial remainder to decide the next add/subtract operation. Therefore, non-restoring division generally reduces corrective work and is attractive in high-speed hardware implementations.

5)A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

field. The given number is -13.25 .

Step 1 – Convert -13.25 to Binary:

$$13 = 1101_2$$

$$0.25 = 0.01_2$$

$$13.25 = 1101.01_2$$

Therefore, $-13.25 = -1101.01_2$

Normalize:

$$-1101.01_2 = -1.10101_2 \times 2^3$$

A. IEEE 754 Single Precision (32-bit)

Single precision contains 1 sign bit, 8 exponent bits, and 23 fraction bits. The sign bit is 1 because the number is negative. The exponent is $3 + 127 = 130 = 10000010_2$. The fraction stores the bits after the leading 1: 10101 followed by zeros.

Sign = 1

Exponent = 10000010

Fraction = 101010000000000000000000

Complete representation:

1 | 10000010 | 101010000000000000000000

Hexadecimal form = C1540000

B. IEEE 754 Double Precision (64-bit)

Double precision contains 1 sign bit, 11 exponent bits, and 52 fraction bits. The exponent bias is 1023, so the stored exponent is $3 + 1023 = 1026 = 10000000010_2$.

Sign = 1

Exponent = 10000000010

Fraction = 10101 followed by 47 zeros

Complete representation:

```
1 | 10000000010 | 1010100000000000000000000000000000000000000000000
```

Hexadecimal form = C02A800000000000

C. Floating-Point Addition

Floating-point addition is performed using the following sequence: (1) compare exponents, (2) shift the significand of the number with the smaller exponent to align the binary points, (3) add or subtract the significands according to their signs, (4) normalize the result, (5) round it to the available precision, and (6) check for exponent overflow or underflow.

Example: $-13.25 + 5.5$

$$-13.25 = -1.10101_2 \times 2^3$$

$$+5.5 = +1.011_2 \times 2^2$$

Align exponent:

$$+5.5 = +0.1011_2 \times 2^3$$

Add significands:

$$-1.10101 + 0.10110 = -0.11111_2 \times 2^3$$

Normalize:

$$-0.11111 \times 2^3 = -1.1111_2 \times 2^2 = -7.75$$

This particular example is exactly representable, so no rounding error occurs. In general, if significant bits are discarded after alignment or normalization, IEEE 754 applies a rounding mode (commonly round-to-nearest, ties-to-even). When two numbers have very different exponents, the smaller number may lose significant bits during alignment, causing precision loss.

Engineering Interpretation: Single precision provides about 7 decimal digits of precision, while double precision provides about 15–16 decimal digits. Double precision is therefore preferred for scientific and engineering calculations requiring greater numerical accuracy, while single precision is often selected when memory usage and computational throughput are more important.